

Project 4: The Serverless Logic

Deploying a Cost Calculator API with AWS Lambda, No Servers Required

Submitter Name:	Khadija Ashraf
Course / Module:	DecodeLabs Cloud Computing Internship
Date Submitted:	18 th September 2026
Cloud Service Used:	AWS Lambda (Serverless / FaaS)
Runtime / Language:	Python 3.x

1. Objective

A company wants to build a simple 'Cost Calculator' API, but running a server 24/7 for such a small, unpredictable workload is wastefully expensive. This project demonstrates deploying code without managing any infrastructure by using AWS Lambda, a serverless compute service. The function accepts two numbers as input and returns their sum, executing only when triggered and costing nothing when idle.

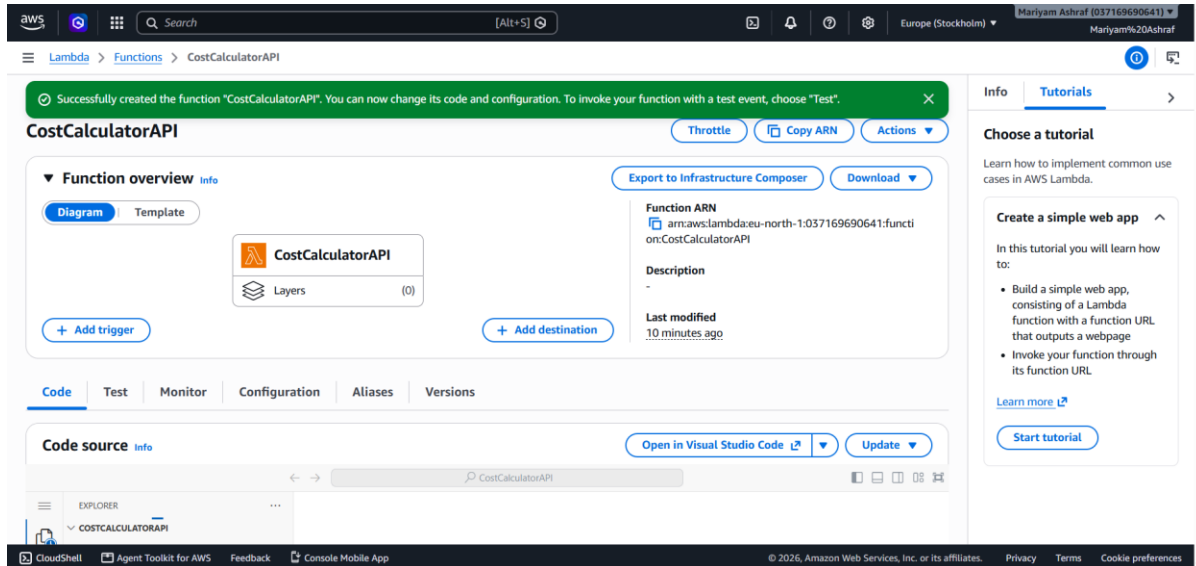
2. Tools & Concepts Used

- Cloud Service: AWS Lambda (Function-as-a-Service / FaaS)
- Language: Python 3.x
- IAM Execution Role: Auto-generated basic execution role with CloudWatch Logs permission
- Testing Method: AWS Console 'Test' tab with mock JSON events
- Monitoring: Amazon CloudWatch Logs

3. Step 1 — Create the Lambda Function

A new Lambda function was created from scratch in the AWS Console, using the Python 3.x runtime with default settings. No servers, operating systems, or scaling infrastructure were provisioned manually.

Screenshot 1: Lambda Function Created

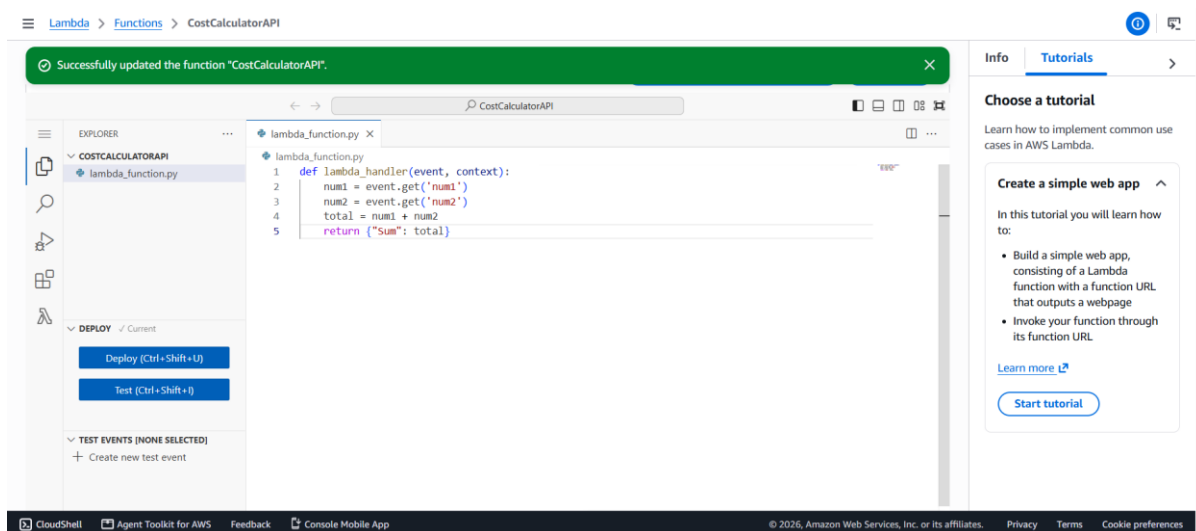


4. Step 2 — Write the Function Code

The following handler function was written in the inline code editor. It extracts two integers from the incoming event payload, calculates their sum, and returns a JSON-serializable dictionary.

```
def lambda_handler(event, context):
    num1 = event.get('num1')
    num2 = event.get('num2')
    total = num1 + num2
    return {"Sum": total}
```

Screenshot 2: Function Code in the Editor



5. Step 3 — Create a Test Event

To simulate a real API call without building a front-end, a private test event was created in the Lambda console's Test tab with a sample JSON payload.

```
{
  "num1": 15,
  "num2": 25
}
```

Screenshot 3: Test Event Configuration

Test event [Info](#) [CloudWatch Logs Live Tail](#) [Save](#) [Test](#)

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

☒ Create new event ☐ Edit saved event

Invocation type

☒ Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

TestEvent1

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

► Generate with Amazon Q

Event JSON [Format JSON](#)

```
1 {
2   "num1": 15,
3   "num2": 25
4 }
```

6. Step 4 — Invoke and Verify the Result

The Test button was clicked to invoke the function with the mock event. The execution result confirmed the function correctly computed the sum and returned it as JSON.

```
Response: {
  "Sum": 40
}
```

Screenshot 4: Successful Test Execution

✔ Executing function: succeeded (logs [↗](#))

▼ Details

Response

Format JSON

```
{
  "Sum": 40
}
```

Summary

Code SHA-256 NTtVNemj3AdoNwejCwsGAs6XVrJJGP2NFQGUkXwaffc=	Execution time 11 seconds ago
Function version \$LATEST	Request ID 3de977a7-a920-4bfd-906b-da5e83cfd769
Duration 1.76 ms	Billed duration 110 ms
Resources configured 128 MB	Max memory used 39 MB
Init duration 107.25 ms	

Log output

The area below shows the last 4 KB of the execution log. [Click here ↗](#) to view the corresponding CloudWatch log group.

```
START RequestId: 3de977a7-a920-4bfd-906b-da5e83cfd769 Version: $LATEST
END RequestId: 3de977a7-a920-4bfd-906b-da5e83cfd769
REPORT RequestId: 3de977a7-a920-4bfd-906b-da5e83cfd769  Duration: 1.76 ms    Billed Duration: 110 ms Memory Size: 128 MB    Max Memory Used: 39 MB Init Duration: 107.25 ms
```

7. Step 5 — Test with a Second, Different Input

To confirm the function's accuracy across different inputs (not just one hardcoded case), a second test event with different values was run.

```
{
  "num1": 100,
  "num2": 250
}

# Expected Response:
{
  "Sum": 350
}
```

Screenshot 5: Second Test with Different Values

Test event Info

CloudWatch Logs Live Tail

Save

Test

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

Create new event

Edit saved event

Invocation type

Synchronous

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

Asynchronous

Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

TestEvent2

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

Private

This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

Shareable

This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

Generate with Amazon Q

Event JSON

Format JSON

1 {

2 "num1": 100,

3 "num2": 250

4 }

Executing function: succeeded [\(logs\)](#)

Details

Response

Format JSON

{

"Sum": 350

}

Summary

Code SHA-256

NTtVNemj3AdoNwejCwsGAs6XvrJJGP2NFQGUkXwaffc=

Execution time

31 seconds ago

Function version

\$LATEST

Request ID

b7f7a9f1-ed0b-41f2-9467-8ff65a9907ba

Duration

40.53 ms

Billed duration

41 ms

Resources configured

128 MB

Max memory used

39 MB

Log output

The area below shows the last 4 KB of the execution log. [Click here](#) to view the corresponding CloudWatch log group.

START RequestId: b7f7a9f1-ed0b-41f2-9467-8ff65a9907ba Version: \$LATEST

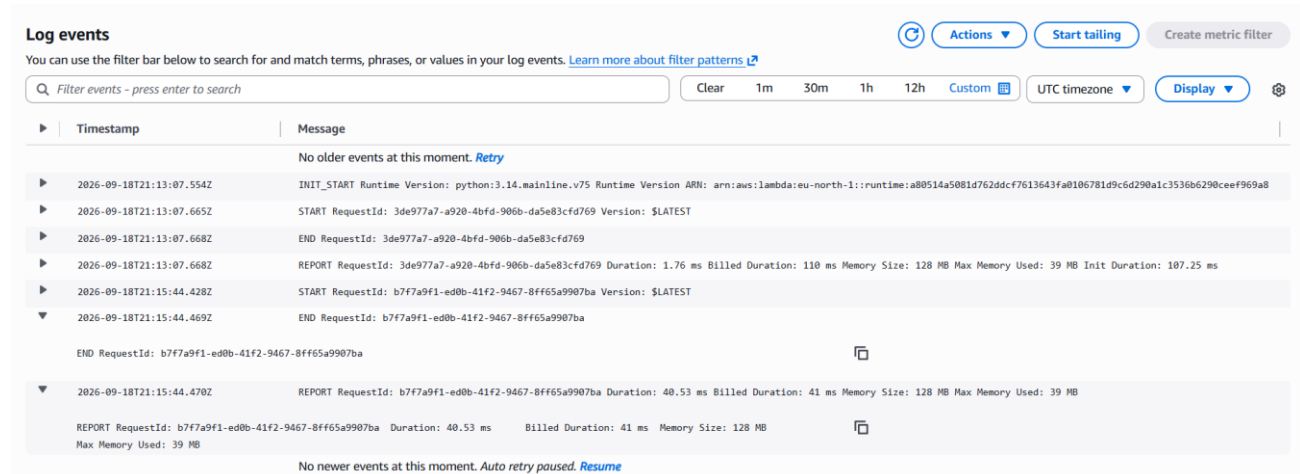
END RequestId: b7f7a9f1-ed0b-41f2-9467-8ff65a9907ba

REPORT RequestId: b7f7a9f1-ed0b-41f2-9467-8ff65a9907ba Duration: 40.53 ms Billed Duration: 41 ms Memory Size: 128 MB Max Memory Used: 39 MB

8. Step 6 — Verify Billing and Execution Evidence in CloudWatch

Every invocation leaves a permanent record in Amazon CloudWatch Logs, proving the exact millisecond duration the function was billed for, demonstrating the cost-efficiency of the serverless model.

Screenshot 6: CloudWatch Log Entry

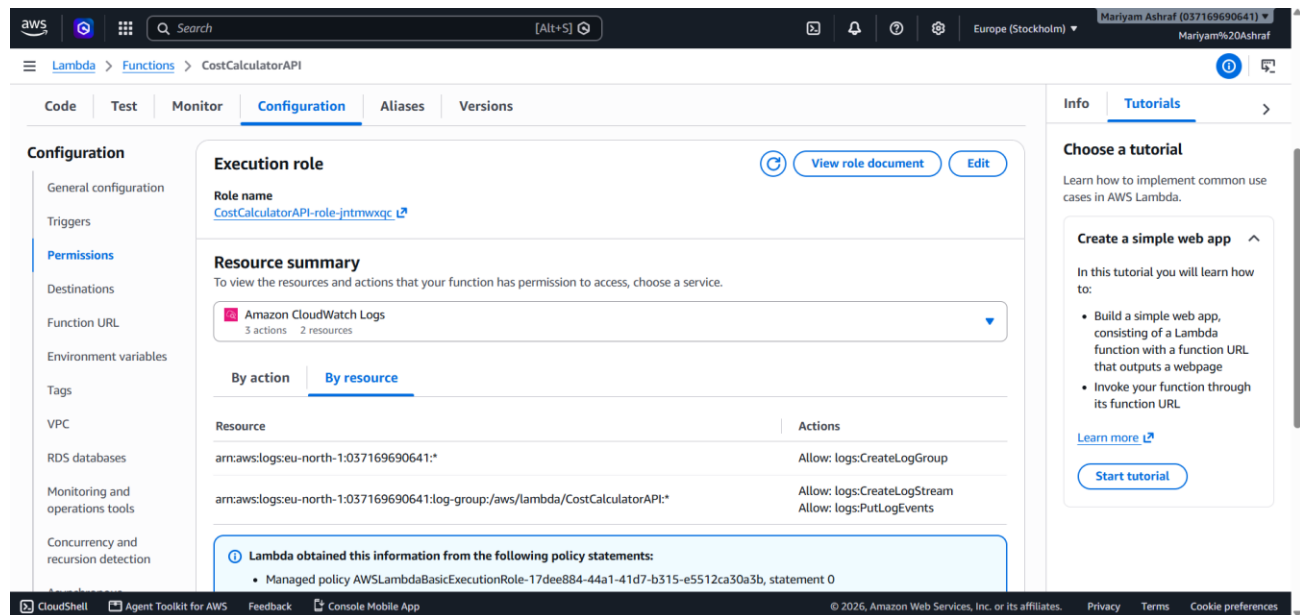


The screenshot shows the AWS CloudWatch 'Log events' console. At the top, there's a search bar with the placeholder 'Filter events - press enter to search'. To the right of the search bar are buttons for 'Clear', '1m', '30m', '1h', '12h', 'Custom', 'UTC timezone', and 'Display'. Below the search bar, there's a table with two columns: 'Timestamp' and 'Message'. The table contains several log entries, including 'INIT_START', 'START', 'END', and 'REPORT' events. Each entry includes a timestamp and a detailed message about the function's execution, including request ID, duration, memory size, and max memory used. For example, one entry shows a 'REPORT' event with a duration of 40.53 ms and a billed duration of 41 ms. At the bottom of the table, there's a message: 'No newer events at this moment. Auto retry paused. Resume'.

9. Step 7 — Review the IAM Execution Role

By default, Lambda functions have zero permissions. AWS automatically attached a basic IAM Execution Role to this function, granting only the minimal permission needed to write logs to CloudWatch, following the Principle of Least Privilege.

Screenshot 7: IAM Execution Role



The screenshot shows the AWS IAM console, specifically the 'Configuration' tab for a Lambda function named 'CostCalculatorAPI'. The 'Execution role' section is highlighted, showing the role name 'CostCalculatorAPI-role-jntmwxq'. Below this, the 'Resource summary' section lists the resources the function has permission to access, including 'Amazon CloudWatch Logs'. The 'By action' tab is selected, showing a table of actions and resources. The table has two columns: 'Resource' and 'Actions'. The first row shows the resource 'arn:aws:logs:eu-north-1:037169690641:*' with the action 'Allow: logs:CreateLogGroup'. The second row shows the resource 'arn:aws:logs:eu-north-1:037169690641:log-group:/aws/lambda/CostCalculatorAPI:*' with the actions 'Allow: logs:CreateLogStream' and 'Allow: logs:PutLogEvents'. At the bottom, there's a note: 'Lambda obtained this information from the following policy statements: Managed policy AWSLambdaBasicExecutionRole-17dee884-44a1-41d7-b315-e5512ca30a3b, statement 0'. On the right side of the console, there's a 'Choose a tutorial' section with a link to 'Create a simple web app'.

10. Conclusion

This project demonstrated the core principle of serverless computing: deploying functional application logic without provisioning, patching, or managing any underlying infrastructure. The Cost Calculator API executes in milliseconds, scales automatically from zero to any volume of requests, and costs nothing while idle, directly solving the 'wasted budget' problem of running a traditional always-on virtual machine for a lightweight, event-driven workload.